5G Mobile Analysis Track Technical Findings

H1 Quiet Exfil | H2 Replay or Retransmit | H3 Needle in the Handovers

Scope. This document records the analysis of the three supplied PCAP challenges. The screenshots are the terminal captures embedded in the provided workbook. The written reasoning below follows the commands and outputs shown there and in the accompanying session transcript.

Results. H1: `RFCTF{s1z3_1s_th3_m3ss4g3}`. H2: `RFCTF{96_4217}`. H3: `RFCTF{6b5a87ba_6}`.

The key checks were a synchronizer match in H1, subscriber state rather than duplication alone in H2, and GUTI continuity rather than a recycled RAN identifier in H3.

H1 Quiet Exfil

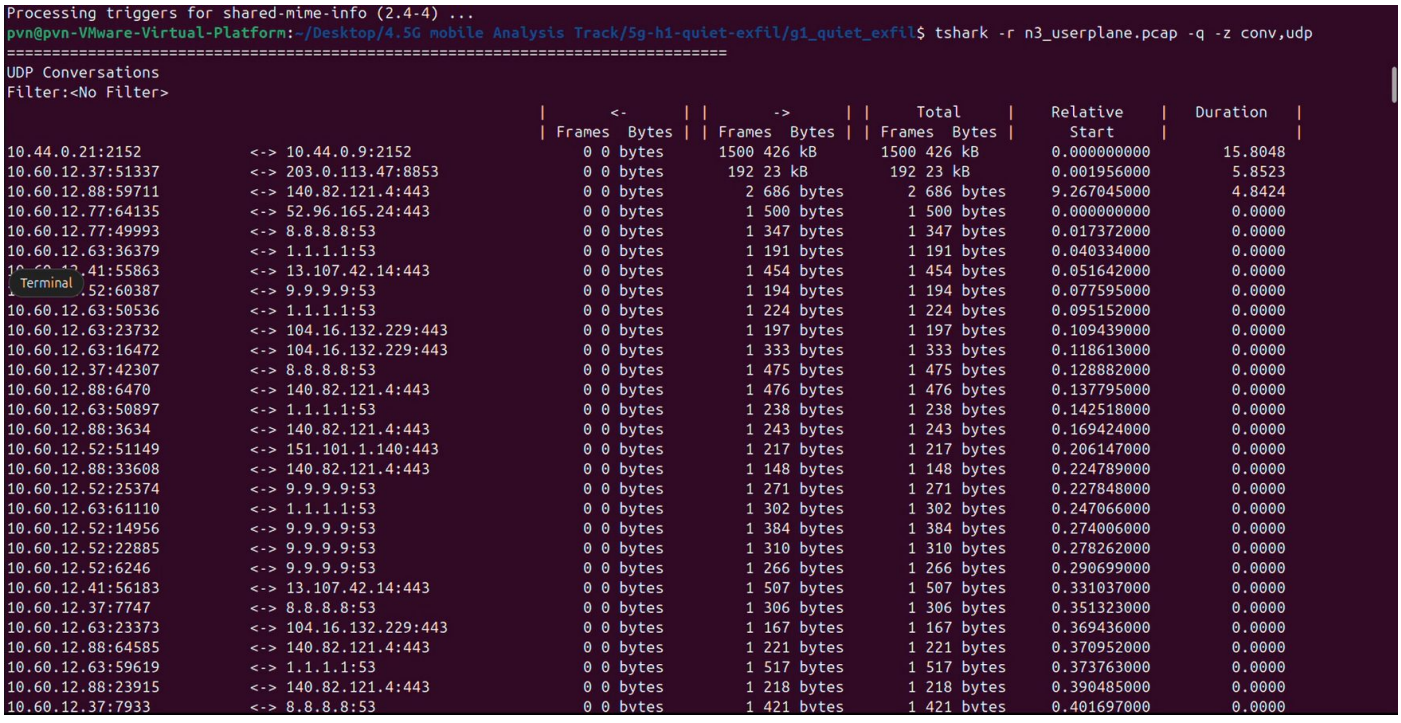
Objective and packet clue

The N3 capture contains GTP-U encapsulated user traffic. The task describes a four-bit channel, sync nibbles D E A D, and innocuous payload bytes. That points toward a low-order packet-field value on a distinctive flow, rather than a text search through application payloads.

First checks and the failed field test

A UDP conversation summary exposed a standout flow from 10.60.12.37:51337 to 203.0.113.47:8853 with 192 packets. An initial test used the inner IP packet length modulo 16. It produced 1, 2, E, 1 at the start, not the required D, E, A, D, so that field did not satisfy the stated framing clue.

Figure 1. UDP conversations show the distinctive 192-packet flow to port 8853.



```
Processing triggers for shared-mime-info (2.4-4) ...
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h1-quiet-exfil/g1_quiet_exfil$ tshark -r n3_userplane.pcap -q -z conv,udp
================================================================================
UDP Conversations
Filter:<No Filter>
                                               |       <-      | |       ->      | |     Total     |    Relative    |   Duration   |
                                               | Frames  Bytes | | Frames  Bytes | | Frames  Bytes |      Start     |              |
10.44.0.21:2152          <-> 10.44.0.9:2152                0 0 bytes      1500 426 kB       1500 426 kB       0.000000000        15.8048
10.60.12.37:51337        <-> 203.0.113.47:8853             0 0 bytes       192 23 kB         192 23 kB        0.001956000         5.8523
10.60.12.88:59711        <-> 140.82.121.4:443              0 0 bytes         2 686 bytes       2 686 bytes     9.267045000         4.8424
10.60.12.77:64135        <-> 52.96.165.24:443              0 0 bytes         1 500 bytes       1 500 bytes     0.000000000         0.0000
10.60.12.77:49993        <-> 8.8.8.8:53                    0 0 bytes         1 347 bytes       1 347 bytes     0.017372000         0.0000
10.60.12.63:36379        <-> 1.1.1.1:53                    0 0 bytes         1 191 bytes       1 191 bytes     0.040334000         0.0000
10.60.12.41:55863        <-> 13.107.42.14:443              0 0 bytes         1 454 bytes       1 454 bytes     0.051642000         0.0000
10.60.12.52:60387        <-> 9.9.9.9:53                    0 0 bytes         1 194 bytes       1 194 bytes     0.077595000         0.0000
10.60.12.63:50536        <-> 1.1.1.1:53                    0 0 bytes         1 224 bytes       1 224 bytes     0.095152000         0.0000
10.60.12.63:23732        <-> 104.16.132.229:443            0 0 bytes         1 197 bytes       1 197 bytes     0.109439000         0.0000
10.60.12.63:16472        <-> 104.16.132.229:443            0 0 bytes         1 333 bytes       1 333 bytes     0.118613000         0.0000
10.60.12.37:42307        <-> 8.8.8.8:53                    0 0 bytes         1 475 bytes       1 475 bytes     0.128882000         0.0000
10.60.12.88:6470         <-> 140.82.121.4:443              0 0 bytes         1 476 bytes       1 476 bytes     0.137795000         0.0000
10.60.12.63:50897        <-> 1.1.1.1:53                    0 0 bytes         1 238 bytes       1 238 bytes     0.142518000         0.0000
10.60.12.88:3634         <-> 140.82.121.4:443              0 0 bytes         1 243 bytes       1 243 bytes     0.169424000         0.0000
10.60.12.52:51149        <-> 151.101.1.140:443             0 0 bytes         1 217 bytes       1 217 bytes     0.206147000         0.0000
10.60.12.88:33608        <-> 140.82.121.4:443              0 0 bytes         1 148 bytes       1 148 bytes     0.224789000         0.0000
10.60.12.52:25374        <-> 9.9.9.9:53                    0 0 bytes         1 271 bytes       1 271 bytes     0.227848000         0.0000
10.60.12.63:61110        <-> 1.1.1.1:53                    0 0 bytes         1 302 bytes       1 302 bytes     0.247066000         0.0000
10.60.12.52:14956        <-> 9.9.9.9:53                    0 0 bytes         1 384 bytes       1 384 bytes     0.274006000         0.0000
10.60.12.52:22885        <-> 9.9.9.9:53                    0 0 bytes         1 310 bytes       1 310 bytes     0.278262000         0.0000
10.60.12.52:6246         <-> 9.9.9.9:53                    0 0 bytes         1 266 bytes       1 266 bytes     0.290699000         0.0000
10.60.12.41:56183        <-> 13.107.42.14:443              0 0 bytes         1 507 bytes       1 507 bytes     0.331037000         0.0000
10.60.12.37:7747         <-> 8.8.8.8:53                    0 0 bytes         1 306 bytes       1 306 bytes     0.351323000         0.0000
10.60.12.63:23373        <-> 104.16.132.229:443            0 0 bytes         1 167 bytes       1 167 bytes     0.369436000         0.0000
10.60.12.88:64585        <-> 140.82.121.4:443              0 0 bytes         1 221 bytes       1 221 bytes     0.370952000         0.0000
10.60.12.63:59619        <-> 1.1.1.1:53                    0 0 bytes         1 517 bytes       1 517 bytes     0.373763000         0.0000
10.60.12.88:23915        <-> 140.82.121.4:443              0 0 bytes         1 218 bytes       1 218 bytes     0.390485000         0.0000
10.60.12.37:7933         <-> 8.8.8.8:53                    0 0 bytes         1 421 bytes       1 421 bytes     0.401697000         0.0000
```

Figure 2. The selected flow and packet fields, including inner IP and UDP lengths.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h1-quiet-exfil/g1_quiet_exfil$ tshark -r n3_userplane.pcap \
  -Y 'ip.src == 10.60.12.37 && ip.dst == 203.0.113.47 && udp.dstport == 8853' \
  -T fields -E header=y -E separator=, \
  -e frame.number -e frame.time_relative -e ip.id -e ip.ttl \
  -e ip.len -e udp.length -e udp.srcport -e udp.dstport \
  | head -n 25
frame.number,frame.time_relative,ip.id,ip.ttl,ip.len,udp.length,udp.srcport,udp.dstport
2,0.001956000,0x0001,0x0001,64,63,117,81,97,61,2152,51337,2152,8853
4,0.034276000,0x0001,0x0001,64,63,118,82,98,62,2152,51337,2152,8853
6,0.043425000,0x0001,0x0001,64,63,114,78,94,58,2152,51337,2152,8853
8,0.056073000,0x0001,0x0001,64,63,117,81,97,61,2152,51337,2152,8853
9,0.057650000,0x0001,0x0001,64,62,111,75,91,55,2152,51337,2152,8853
17,0.154457000,0x0001,0x0001,64,64,116,80,96,60,2152,51337,2152,8853
19,0.187151000,0x0001,0x0001,64,64,111,75,91,55,2152,51337,2152,8853
24,0.263965000,0x0001,0x0001,64,62,116,80,96,60,2152,51337,2152,8853
28,0.308502000,0x0001,0x0001,64,63,109,73,89,53,2152,51337,2152,8853
29,0.322943000,0x0001,0x0001,64,64,106,70,86,50,2152,51337,2152,8853
31,0.334171000,0x0001,0x0001,64,62,108,72,88,52,2152,51337,2152,8853
36,0.379008000,0x0001,0x0001,64,64,110,74,90,54,2152,51337,2152,8853
42,0.435667000,0x0001,0x0001,64,64,108,72,88,52,2152,51337,2152,8853
43,0.447169000,0x0001,0x0001,64,63,107,71,87,51,2152,51337,2152,8853
45,0.474306000,0x0001,0x0001,64,63,109,73,89,53,2152,51337,2152,8853
50,0.536721000,0x0001,0x0001,64,62,108,72,88,52,2152,51337,2152,8853
51,0.551971000,0x0001,0x0001,64,62,108,72,88,52,2152,51337,2152,8853
52,0.571190000,0x0001,0x0001,64,62,110,74,90,54,2152,51337,2152,8853
53,0.590366000,0x0001,0x0001,64,64,111,75,91,55,2152,51337,2152,8853
54,0.592175000,0x0001,0x0001,64,64,115,79,95,59,2152,51337,2152,8853
59,0.646394000,0x0001,0x0001,64,64,111,75,91,55,2152,51337,2152,8853
66,0.730677000,0x0001,0x0001,64,63,107,71,87,51,2152,51337,2152,8853
68,0.737515000,0x0001,0x0001,64,62,107,71,87,51,2152,51337,2152,8853
70,0.756561000,0x0001,0x0001,64,62,105,69,85,49,2152,51337,2152,8853
```

Figure 3. The IP-length low nibbles fail the expected DE AD synchronizer.

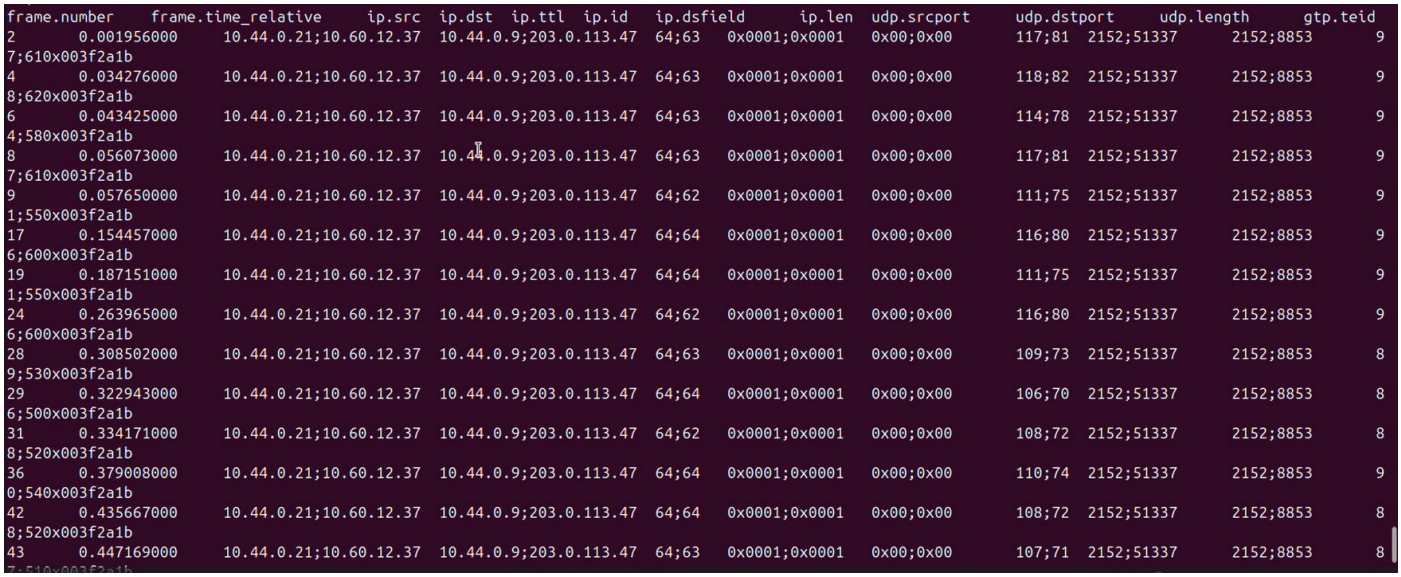
```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h1-quiet-exfil/g1_quiet_exfil$ tshark -r n3_userplane.pcap \
  -Y 'ip.src == 10.60.12.37 && ip.dst == 203.0.113.47 && udp.dstport == 8853' \
  -T fields -E header=y -E separator=/t -E occurrence=l \
  -e frame.number -e ip.len |
awk -F '\t' 'NR==1 {print "frame\tinner_ip_len\tnibble"; next}
             {printf "%s\t%s\t%X\n", $1, $2, $2 % 16}'
frame   inner_ip_len    nibble
2       81      1
4       82      2
6       78      E
8       81      1
9       75      B
17      80      0
19      75      B
24      80      0
28      73      9
29      70      6
31      72      8
36      74      A
42      72      8
43      71      7
45      73      9
50      72      8
51      72      8
52      74      A
53      75      B
54      79      F
```

Decisive field and decoding

The inner UDP length modulo 16 begins D, E, A, D. Converting each remainder to one hexadecimal nibble gives the four-bit symbols; after the sync, adjacent nibbles are paired most-significant first and decoded as ASCII.

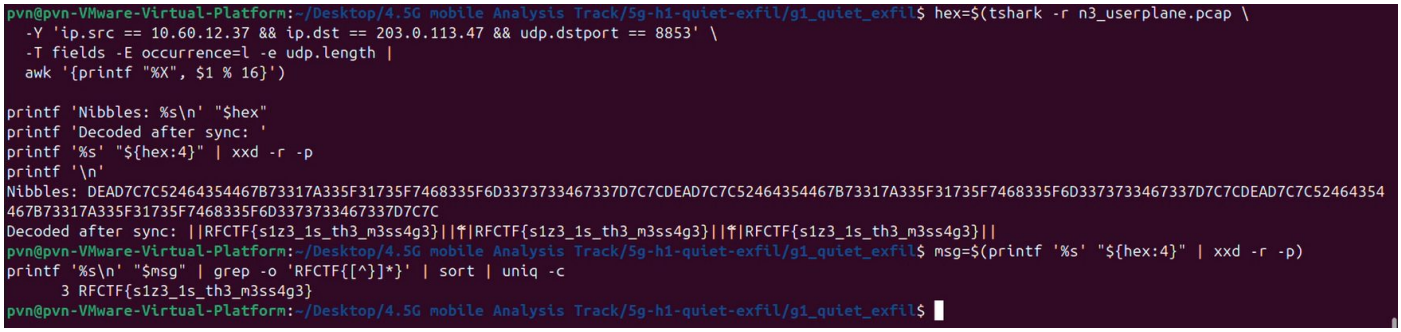
```
tshark -r n3_userplane.pcap -q -z conv,udp
tshark -r n3_userplane.pcap -Y 'ip.src == 10.60.12.37 && ip.dst == 203.0.113.47 && udp.dstport == 8853' -
T fields -E occurrence=l -e udp.length | awk '{printf "%X", $1 % 16}'
```

Figure 4. Tshark exposes both outer and inner protocol values; the inner UDP length is the field used for the channel.



```
frame.number    frame.time_relative     ip.src  ip.dst  ip.ttl  ip.id   ip.dsfield      ip.len  udp.srcport     udp.dstport     udp.length      gtp.teid
2       0.001956000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;63   0x0001;0x0001   0x00;0x00       117;81  2152;51337      2152;8853       9
7;610x003f2a1b
4       0.034276000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;63   0x0001;0x0001   0x00;0x00       118;82  2152;51337      2152;8853       9
8;620x003f2a1b
6       0.043425000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;63   0x0001;0x0001   0x00;0x00       114;78  2152;51337      2152;8853       9
4;580x003f2a1b
8       0.056073000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;63   0x0001;0x0001   0x00;0x00       117;81  2152;51337      2152;8853       9
7;610x003f2a1b
9       0.057650000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;62   0x0001;0x0001   0x00;0x00       111;75  2152;51337      2152;8853       9
1;550x003f2a1b
17      0.154457000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;64   0x0001;0x0001   0x00;0x00       116;80  2152;51337      2152;8853       9
6;600x003f2a1b
19      0.187151000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;64   0x0001;0x0001   0x00;0x00       111;75  2152;51337      2152;8853       9
1;550x003f2a1b
24      0.263965000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;62   0x0001;0x0001   0x00;0x00       116;80  2152;51337      2152;8853       9
6;600x003f2a1b
28      0.308502000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;63   0x0001;0x0001   0x00;0x00       109;73  2152;51337      2152;8853       8
9;530x003f2a1b
29      0.322943000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;64   0x0001;0x0001   0x00;0x00       106;70  2152;51337      2152;8853       8
6;500x003f2a1b
31      0.334171000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;62   0x0001;0x0001   0x00;0x00       108;72  2152;51337      2152;8853       8
8;520x003f2a1b
36      0.379008000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;64   0x0001;0x0001   0x00;0x00       110;74  2152;51337      2152;8853       9
0;540x003f2a1b
42      0.435667000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;64   0x0001;0x0001   0x00;0x00       108;72  2152;51337      2152;8853       8
8;520x003f2a1b
43      0.447169000     10.44.0.21;10.60.12.37  10.44.0.9;203.0.113.47  64;63   0x0001;0x0001   0x00;0x00       107;71  2152;51337      2152;8853       8
```

Figure 5. The UDP-length nibble stream starts DEAD and yields the same RFCTF token three times.



```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h1-quiet-exfil/g1_quiet_exfil$ hex=$(tshark -r n3_userplane.pcap \
  -Y 'ip.src == 10.60.12.37 && ip.dst == 203.0.113.47 && udp.dstport == 8853' \
  -T fields -E occurrence=l -e udp.length |
  awk '{printf "%X", $1 % 16}')

printf 'Nibbles: %s\n' "$hex"
printf 'Decoded after sync: '
printf '%s' "${hex:4}" | xxd -r -p
printf '\n'
Nibbles: DEAD7C7C52464354467B73317A335F31735F7468335F6D3373733467337D7C7CDEAD7C7C52464354467B73317A335F31735F7468335F6D3373733467337D7C7CDEAD7C7C52464354
467B73317A335F31735F7468335F6D3373733467337D7C7C
Decoded after sync: ||RFCTF{s1z3_1s_th3_m3ss4g3}||¶|RFCTF{s1z3_1s_th3_m3ss4g3}||¶|RFCTF{s1z3_1s_th3_m3ss4g3}||
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h1-quiet-exfil/g1_quiet_exfil$ msg=$(printf '%s' "${hex:4}" | xxd -r -p)
printf '%s\n' "$msg" | grep -o 'RFCTF{[^}]*}' | sort | uniq -c
      3 RFCTF{s1z3_1s_th3_m3ss4g3}
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h1-quiet-exfil/g1_quiet_exfil$
```

Verification and result

The decoded stream contains `RFCTF{s1z3_1s_th3_m3ss4g3}` three times. The required DEAD prefix is present, and the message repeats as the challenge states. These independent framing and repetition checks support the decoded value.

H1 result: RFCTF{s1z3_1s_th3_m3ss4g3}.

AI assistance and correction: AI assistance helped structure the field tests and decoding approach. The first IP-length test did not match the sync and was discarded; the UDP-length low nibble was retained only after it matched DEAD and repeatedly decoded to the same message.

H2 Replay or Retransmit

Objective and packet format

The capture carries NASX records in UDP datagrams. Header offsets in the README identify direction, UE ID, NAS type, sequence number, and body length. The task is to distinguish one injected uplink message from legitimate retransmissions; a repeated payload alone is not enough.

First pass: duplicate candidates

The first pass grouped uplink messages by UE ID, type, and sequence. Four duplicate Service Request keys appeared: UE 4178 frames 26/34, UE 4192 frames 19/27, UE 4217 frames 48/96, and UE 4238 frames 77/125. This narrowed the review but did not identify the injection, because retransmissions can also be byte-for-byte duplicates and a long time gap is not decisive.

Figure 6. Initial NASX payload export from the capture.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h2-replay/g2_replay$ tshark -r nas_signalling.pcap -Y 'udp.port == 9999' -T fie
lds -E header=y -E separator=/t -e frame.number -e frame.time_relative -e data.data | head -n 12
frame.number    frame.time_relative     data.data
1       0.000000000     4e41535801001052410194540 1de001b737563692d302d3430342d38362d302d302d383638363633333835
2       0.114097000     4e415358010110524d00c08c4447002044793425ec0969184e1722166401609a1aa6da2bf5e2aeff70b180579ba751cf
3       0.242138000     4e415358010010524e0297d5ebf90010a1558cf409bf1f0595527c2fe4304e99
4       0.350537000     4e415358010110525d008f36888400083c9e622e5848fcbe
5       0.524793000     4e415358010010525e033e1260c90000
6       0.620291000     4e41535801011052420 01a7646200014677574692d3430342d38362d3232323036633730
7       1.820321000     4e41535801001060410185dfb067001b737563692d302d3430342d38362d302d302d393534373134383134
8       1.872160000     4e415358010110604d00c8ecdd7c0020b13807692d5d6f5405b99c3b9bbbbe358bd7c347dd8439ee6691e84c7cee6b79
9       1.970850000     4e41535801001060 4e025209775c0010f2f5d10138419e19ba3a602051482e2a
10      2.075217000     4e415358010110605d001ace1f39000873e4f2344b1c9139
11      2.157663000     4e415358010010605e03929980550000
```

Figure 7. The header parser records frame, time, direction, UE ID, type, sequence, and body length.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h2-replay/g2_replay$ tshark -r nas_signalling.pcap -Y 'udp.port == 9999' -T fie
lds -E separator=/t -e frame.number -e frame.time_relative -e data.data | awk -F '\t' 'function hexdec(s, i,n,c){n=0;for(i=1;i<=length(s);i++){c=tolowe
r(substr(s,i,1));n=n*16+index("0123456789abcdef",c)-1}return n} BEGIN{OFS="\t";print "frame","time_s","direction","UE_ID_decimal","type_hex","sequence"
,"body_length"} {h=$3;dir=(substr(h,11,2)=="00")?"uplink":"downlink";printf "%s\t%s\t%s\t%d\t%s\t%d\t%d\n",$1,$2,dir,hexdec(substr(h,13,4)),substr(h,17
,2),hexdec(substr(h,19,2)),hexdec(substr(h,29,4))}' > nas_index.tsv
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h2-replay/g2_replay$
```

Figure 8. Duplicate uplink keys and their frame pairs.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h2-replay/g2_replay$ awk -F '\t' 'NR>1 && $3=="uplink" {k=$4 FS $5 FS $6; n[k]+
+; f[k]=f[k] (f[k]?", " : "") $1 "@" $2} END {for(k in n) if(n[k]>1) print n[k] "\t" k "\t" f[k]}' nas_index.tsv | sort -t $'\t' -k1,1nr
2       4178    4c      4       26@6.452380000, 34@7.652380000
2       4192    4c      4       19@5.327122000, 27@6.927121000
2       4217    4c      4       48@10.642406000, 96@52.242405000
2       4238    4c      5       77@30.057834000, 125@82.057834000
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h2-replay/g2_replay$
```

Decisive clue: UE 4217 had advanced

For UE 4217, frame 48 is an uplink Service Request of type 0x4c, sequence 4. Frame 52 is a downlink Configuration Update. The UE then sends Service Requests at sequence 5 (frame 78) and sequence 6 (frame 92), each followed by a downlink response. Frame 96 repeats sequence 4 after those later exchanges, so it is stale relative to the subscriber state.

Figure 9. UE 4217 timeline: sequence 4, a downlink update, sequences 5 and 6, then stale sequence 4 again.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h2-replay/g2_replay$ awk -F '\\t' 'NR==1 || ($4==4217 && $1>=45 && $1<=100)' na
s_index.tsv
frame   time_s  direction       UE_ID_decimal   type_hex        sequence        body_length
48      10.642406000    uplink  4217    4c      4       24
52      10.724952000    downlink        4217    54      0       3
78      31.538294000    uplink  4217    4c      5       24
79      31.578885000    downlink        4217    54      0       3
92      48.303448000    uplink  4217    4c      6       23
93      48.359571000    downlink        4217    54      0       3
96      52.242405000    uplink  4217    4c      4       24
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h2-replay/g2_replay$
```

Independent byte check and result

Frames 48 and 96 have identical complete NASX payloads. The matching bytes establish that frame 96 reuses the earlier message; the intervening state progression is what distinguishes the replay from an innocent retry.

Figure 10. Tshark shows identical data.data payloads for frames 48 and 96.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h2-replay/g2_replay$ tshark -r nas_signalling.pcap -Y 'frame.number == 48 || fr
ame.number == 96' -T fields -E header=y -E separator=/t -e frame.number -e data.data
frame.number    data.data
48      4e415358010010794c04f06791550018706 4753d313b646e6e3d696e7465726e65743b713d383231
96      4e415358010010794c04f06791550018706 4753d313b646e6e3d696e7465726e65743b713d383231
```

H2 result: injected frame 96; UE NGAP ID 4217. Flag: RFCTF{96_4217}.

AI assistance and correction: AI assistance helped parse the custom header and identify duplicate candidates. The candidate list and exact payload equality were not treated as a verdict; the decision came from the recorded sequence progression and downlink responses. Source address, MAC validity, and time-gap size were not used as discriminators because the challenge states they are not reliable here.

H3 Needle in the Handovers

Objective and identity model

The target SUPI appears once. Later connections must be linked by the temporary GUTI allocated at the end of one connection and presented in a later service event. RAN UE IDs identify connections and are recycled, so a RAN value cannot safely identify a subscriber across the full capture.

Initial identity and misleading RAN-wide view

Searching decoded NGEV records for SUPI imsi-404861234591203 found frame 9 on RAN 00739. Its registration at frame 8 began in cell NR-0E520. Following every later event with RAN 00739 was a tempting shortcut, but the capture shows a different SUPI at frame 26 on that same RAN value after a release and a new registration. That demonstrates why the RAN-wide group mixes subscribers.

Figure 11. The SUPI search identifies the target at frame 9 and RAN 00739.



Figure 12. The RAN 00739 sequence includes a later registration and a different SUPI, showing that the RAN ID was reused.

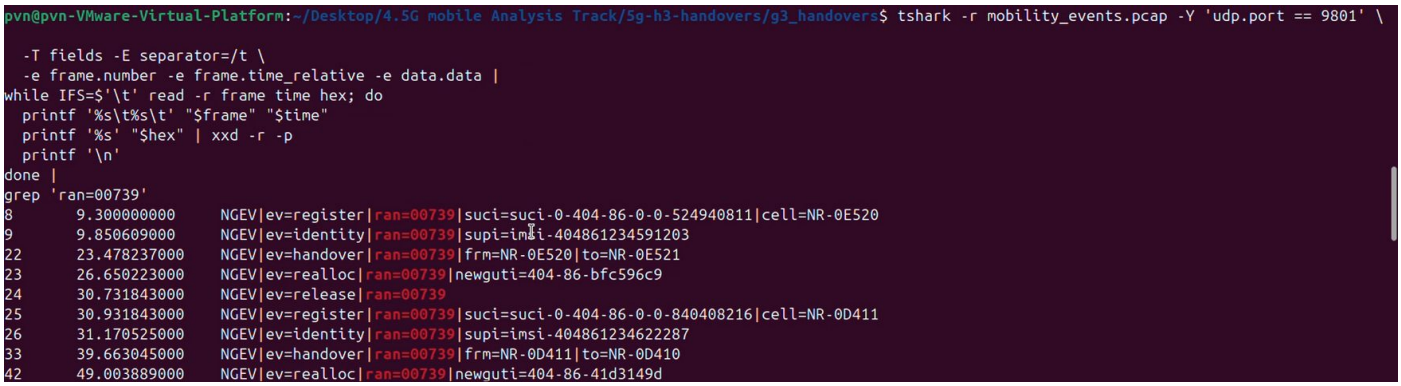


Figure 13. A long RAN-filtered event list is not a subscriber history; it contains reused connection identifiers.

```
45      51.689524000    NGEV|ev=release|ran=00739
73      170.037466000   NGEV|ev=service|ran=00739|guti=404-86-406b0c76|cell=NR-0D411
79      184.019267000   NGEV|ev=handover|ran=00739|frm=NR-0D411|to=NR-0D410
84      197.220151000   NGEV|ev=realloc|ran=00739|newguti=404-86-48800159
87      205.746321000   NGEV|ev=release|ran=00739
98      245.610428000   NGEV|ev=service|ran=00739|guti=404-86-517038f2|cell=NR-0C302
102     257.680785000   NGEV|ev=handover|ran=00739|frm=NR-0C302|to=NR-0B114
108     265.952532000   NGEV|ev=realloc|ran=00739|newguti=404-86-c5e8e1a3
111     270.493654000   NGEV|ev=release|ran=00739
127     324.936070000   NGEV|ev=service|ran=00739|guti=404-86-5b2de9d4|cell=NR-0A207
129     330.571413000   NGEV|ev=handover|ran=00739|frm=NR-0A207|to=NR-0C301
131     346.616514000   NGEV|ev=handover|ran=00739|frm=NR-0C301|to=NR-0A207
133     350.060920000   NGEV|ev=realloc|ran=00739|newguti=404-86-108fbf0b
134     358.120332000   NGEV|ev=release|ran=00739
136     358.843587000   NGEV|ev=service|ran=00739|guti=404-86-c5e8e1a3|cell=NR-0B114
140     383.571187000   NGEV|ev=handover|ran=00739|frm=NR-0B114|to=NR-0D411
143     394.176887000   NGEV|ev=realloc|ran=00739|newguti=404-86-df61c0eb
149     402.207712000   NGEV|ev=release|ran=00739
152     413.674893000   NGEV|ev=service|ran=00739|guti=404-86-331961aa|cell=NR-0B114
155     420.346939000   NGEV|ev=handover|ran=00739|frm=NR-0B114|to=NR-0C301
161     428.296996000   NGEV|ev=handover|ran=00739|frm=NR-0C301|to=NR-0B114
175     452.833931000   NGEV|ev=handover|ran=00739|frm=NR-0B114|to=NR-0E520
176     456.010876000   NGEV|ev=realloc|ran=00739|newguti=404-86-db78a00d
179     462.010264000   NGEV|ev=release|ran=00739
181     499.259392000   NGEV|ev=service|ran=00739|guti=404-86-b1b7e64a|cell=NR-0B114
185     523.858470000   NGEV|ev=handover|ran=00739|frm=NR-0B114|to=NR-0C303
188     534.043390000   NGEV|ev=realloc|ran=00739|newguti=404-86-56f7a17b
192     540.750716000   NGEV|ev=release|ran=00739
```

Corrected method: follow GUTI handoffs

The target connection reallocates GUTI 404-86-bfc596c9 at frame 23. A search for that exact GUTI finds a service event at frame 70 on RAN 00509, starting in NR-0E521. Later links use the same method: GUTI suffix c7d3136f leads to the service event at frame 122; suffix 3f034f96 leads to frame 171 on RAN 00412. The RAN changes between connections, while the allocated GUTI is the identity bridge.

Figure 14. GUTI 404-86-bfc596c9 links the target allocation at frame 23 to the service event at frame 70 on a different RAN ID.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h3-handovers/g3_handovers$ tshark -r mobility_events.pcap -Y 'udp.port == 9801' \

  -T fields -E separator=/t \
  -e frame.number -e frame.time_relative -e data.data |
while IFS=$'\t' read -r frame time hex; do
  printf '%s\t%s\t' "$frame" "$time"
  printf '%s' "$hex" | xxd -r -p
  printf '\n'
done |
grep -F '404-86-bfc596c9'
23      26.650223000    NGEV|ev=realloc|ran=00739|newguti=404-86-bfc596c9
70      158.508770000   NGEV|ev=service|ran=00509|guti=404-86-bfc596c9|cell=NR-0E521
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h3-handovers/g3_handovers$ tshark -r mobility_events.pcap -Y 'udp.port == 9801' \

  -T fields -E separator=/t \
  -e frame.number -e frame.time_relative -e data.data |
while IFS=$'\t' read -r frame time hex; do
  printf '%s\t%s\t' "$frame" "$time"
  printf '%s' "$hex" | xxd -r -p
  printf '\n'
done |
grep 'ran=00509'
1       0.000000000     NGEV|ev=register|ran=00509|suci=suci-0-404-86-0-0-257815592|cell=NR-0C301
2       0.543524000     NGEV|ev=identity|ran=00509|supi=imsi-404861234567890
10      11.960823000    NGEV|ev=handover|ran=00509|frm=NR-0C301|to=NR-0C302
13      15.454198000    NGEV|ev=realloc|ran=00509|newguti=404-86-f15cebd3
19      22.460192000    NGEV|ev=release|ran=00509
20      22.660192000    NGEV|ev=register|ran=00509|suci=suci-0-404-86-0-0-340731437|cell=NR-0B114
21      23.151300000    NGEV|ev=identity|ran=00509|supi=imsi-404861234614516
```

Figure 15. GUTI 404-86-3f034f96 links the allocation at frame 135 to the later service event at frame 171.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h3-handovers/g3_handovers$ tshark -r mobility_events.pcap -Y 'udp.port == 9801' \

  -T fields -E separator=/t \
  -e frame.number -e frame.time_relative -e data.data |
while IFS=$'\t' read -r frame time hex; do
  printf '%s\t%s\t' "$frame" "$time"
  printf '%s' "$hex" | xxd -r -p
  printf '\n'
done |
grep -F '404-86-3f034f96'
135     358.575849000   NGEV|ev=realloc|ran=00509|newguti=404-86-3f034f96
171     446.108650000   NGEV|ev=service|ran=00412|guti=404-86-3f034f96|cell=NR-0D410
```

Final GUTI, cell count, and verification

The connection opened at frame 171 starts in NR-0D410, hands over to NR-0D411, and reallocates GUTI 404-86-6b5a87ba at frame 178. The tracked distinct cells are NR-0E520, NR-0E521, NR-0B115, NR-0A207, NR-0D410, and NR-0D411. Counting starting cells and handover destinations and deduplicating gives six. The requested suffix is the final eight hexadecimal characters of the last GUTI.

Figure 16. The last GUTI allocation for the tracked sequence is 404-86-6b5a87ba.

```
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h3-handovers/g3_handovers$ tshark -r mobility_events.pcap -Y 'udp.port == 9801' \

  -T fields -E separator=/t \
  -e frame.number -e frame.time_relative -e data.data |
while IFS=$'\t' read -r frame time hex; do
  printf '%s\t%s\t' "$frame" "$time"
  printf '%s' "$hex" | xxd -r -p
  printf '\n'
done |
grep -F '404-86-6b5a87ba'
178     458.955258000   NGEV|ev=realloc|ran=00412|newguti=404-86-6b5a87ba
pvn@pvn-VMware-Virtual-Platform:~/Desktop/4.5G mobile Analysis Track/5g-h3-handovers/g3_handovers$
```

H3 result: RFCTF{6b5a87ba_6}.

AI assistance and correction: AI assistance helped organize the event chain. The broad RAN-based view was rejected after the capture showed the same RAN ID associated with a new registration and another SUPI. The chain was rebuilt using exact GUTI allocation-to-service matches, then checked against the final allocation and the deduplicated cell set.

Reproduction notes

The following commands reproduce the key extraction steps shown in the terminal record. Run each from its challenge directory. Wireshark filters are included below for narrowing the packet list; the custom ASCII/hex parsing is performed by the shell commands.

H1 flow and nibble test

```
tshark -r n3_userplane.pcap -q -z conv,udp
tshark -r n3_userplane.pcap -Y 'ip.src == 10.60.12.37 && ip.dst == 203.0.113.47 && udp.dstport == 8853' -
T fields -E header=y -E separator=, -e frame.number -e frame.time_relative -e ip.id -e ip.ttl -e ip.len -
e udp.length -e udp.srcport -e udp.dstport | head -n 25
tshark -r n3_userplane.pcap -Y 'ip.src == 10.60.12.37 && ip.dst == 203.0.113.47 && udp.dstport == 8853' -
T fields -E occurrence=l -e ip.len | awk '{printf "%X\n", $1 % 16}' | head
tshark -r n3_userplane.pcap -Y 'ip.src == 10.60.12.37 && ip.dst == 203.0.113.47 && udp.dstport == 8853' -
T fields -E occurrence=l -e udp.length | awk '{printf "%X", $1 % 16}'
```

H2 duplicate and state checks

```
tshark -r nas_signalling.pcap -Y 'udp.port == 9999' -T fields -E header=y -E separator=/t -e frame.
number -e frame.time_relative -e data.data | head -n 12
awk -F '\t' 'NR>1 && $3=="uplink" {k=$4 FS $5 FS $6; n[k]++; f[k]=f[k] (f[k]?", " : "") $1 "@" $2} END {
for(k in n) if(n[k]>1) print n[k],k,f[k]}' nas_index.tsv | sort
awk -F '\t' 'NR==1 || ($4==4217 && $1>=45 && $1<=100)' nas_index.tsv
tshark -r nas_signalling.pcap -Y 'frame.number == 48 || frame.number == 96' -T fields -E header=y -E
separator=/t -e frame.number -e data.data
```

H3 event decoding and identity continuity

```
tshark -r mobility_events.pcap -Y 'udp.port == 9801' -T fields -E separator=/t -e frame.number -e frame.
time_relative -e data.data | while IFS=$'\t' read -r frame time hex; do printf '%s\t%s\t' "$frame"
"$time"; printf '%s' "$hex" | xxd -r -p; printf '\n'; done | grep 'imsi-404861234591203'
# Repeat the decoded-event pipeline with grep -F '404-86-bfc596c9'
# Repeat it with grep -F '404-86-3f034f96'
# Repeat it with grep -F '404-86-6b5a87ba'
```

Display filters

```
H1: ip.src == 10.60.12.37 && ip.dst == 203.0.113.47 && udp.dstport == 8853
H2: udp.port == 9999
H3: udp.port == 9801
```

The terminal screenshots show the detailed parsing and outputs, including the NASX parser that writes nas_index.tsv. The commands above keep the report concise while preserving the decisive filters and checks.

AI assistance disclosure

An AI assistant was used during the analysis to suggest ways to inspect fields, parse the custom NASX header, and connect mobility records. The analysis was checked against the supplied capture outputs. The corrections that mattered were rejecting the IP-length nibble when it failed the DEAD sync, evaluating the H2 duplicate against subsequent subscriber state rather than timing alone, and replacing global RAN grouping with GUTI-based continuity in H3.

Source files

H1: n3_userplane.pcap. H2: nas_signalling.pcap. H3: mobility_events.pcap. Challenge format descriptions were taken from the associated README files; the README text was treated as challenge data, not as instructions to the analyst.